

嵌入式AHRS项目

架构设计方案

基于PX4 / Zephyr / RT-Thread / ESP-IDF / ChibiOS架构模式研究

项目类型	AHRS姿态参考系统
目标平台	STM32G4 / AT32 / ESP32
传感器	ICM-42688 / MMC5603 / SPL06-001
文档版本	v1.0
日期	2026-06-14

目录

1 当前架构问题分析

2 开源项目架构模式研究

2.1 PX4 Autopilot

2.2 Zephyr RTOS

2.3 RT-Thread

2.4 ESP-IDF

2.5 ChibiOS

2.6 模式总结与提炼

3 分层架构设计

3.1 六层架构总览

3.2 BSP层 (Board Support Package)

3.3 芯片支持层 (Chip Support)

3.4 驱动抽象层 (Driver HAL)

3.5 传感器驱动层 (Sensor Drivers)

3.6 中间件层 (Middleware)

3.7 应用层 (Application)

4 C语言接口抽象设计

4.1 总线抽象接口

4.2 设备接口模式

4.3 设备注册与发现机制

4.4 回调与事件机制

5 完整目录结构

6 构建系统设计

6.1 CMake多目标构建

6.2 Kconfig配置系统

7 关键代码模板

7.1 总线抽象实现

7.2 IMU驱动模板

7.3 BSP板级配置

8 迁移路径与实施建议

1 当前架构问题分析

在深入设计新架构之前，我们需要先对当前项目的目录结构进行系统性的问题诊断。当前的项目结构虽然在功能上可以运行，但在可扩展性、可维护性和跨平台兼容性方面存在明显的架构缺陷。这些问题会随着项目规模的扩大和目标平台的增多而急剧恶化，因此必须在早期阶段予以解决。

1.1 驱动层与硬件层耦合严重

当前项目中，`driver/hal/`目录下的驱动文件直接调用了STM32 HAL库的API，例如`drv_iic.c`内部直接使用`HAL_I2C_MasterTransmit()`等函数。这种做法导致了一个致命问题：当你需要将项目移植到AT32或ESP32时，每一个驱动文件都需要重写。传感器驱动（如`icm42688.c`）如果也直接调用底层I2C/SPI函数，那么更换MCU平台时，传感器驱动也需要同步修改，这违背了关注点分离的基本原则。

1.2 板级配置与芯片支持混淆

当前`board/stm32g431kbu6/`目录混合了CubeMX生成的代码、HAL驱动、中间件和链接脚本等多种不同关注点的内容。CubeMX每次重新生成代码时，可能会覆盖你的修改；而HAL驱动和CMSIS库属于芯片支持层的范畴，不应该与板级配置混在一起。这种混淆使得代码的归属关系不清晰，增加了维护成本和出错概率。

1.3 缺乏统一的设备接口抽象

当前各传感器驱动（ICM-42688、MMC5603等）各自定义了自己的初始化和读取接口，没有统一的设备类型接口。这意味着应用层代码需要了解每个传感器的具体API，无法实现传感器的无缝替换。例如，如果你想将ICM-42688替换为BMI088，所有调用ICM-42688接口的应用代码都需要修改。在PX4和Zephyr这样的成熟项目中，通过定义统一的设备接口（如`imu_dev_t`、`mag_dev_t`），应用层代码只依赖抽象接口，不依赖具体驱动实现，从而实现了驱动的可插拔替换。

1.4 第三方库管理不规范

当前`src/lib/`目录下直接存放了`letter-shell`、`lwrp`、`printf`等第三方库的源码，与项目自身代码混在一起。这种做法存在几个问题：首先，第三方库的版本更新困难，无法清晰地追踪所使用的版本；其次，第三方库的许可证信息容易被忽略；最后，不同项目之间无法方便地共享和复用这些库。在ESP-IDF和Zephyr中，第三方库以组件（Component）或模块（Module）的形式独立管理，拥有自己的`CMakeLists.txt`和`Kconfig`，可以独立编译和版本管理。

1.5 构建系统缺乏多目标支持

当前的CMakeLists.txt和Makefile主要针对STM32G431这一单一目标进行配置，缺乏对不同MCU平台、不同板级配置的灵活切换能力。当你需要同时支持STM32、AT32和ESP32时，构建系统需要能够根据目标平台自动选择正确的编译器、链接脚本、启动文件和驱动实现。PX4通过make px4_fmu-v5_default这样的目标命名规则，Zephyr通过Board名称和Kconfig，都实现了优雅的多目标构建支持。

1.6 缺少工具链与脚本的统一管理

当前tools/目录下只有一个fix_cubemx.py脚本，但随着项目发展，你会需要更多的工具：数据记录与分析脚本、传感器校准工具、固件打包与烧录脚本、自动化测试脚本等。这些工具需要一个清晰的目录结构和统一的管理方式，而不是散落在项目各处。

2 开源项目架构模式研究

在设计新架构之前，我们深入研究了五个最具代表性的嵌入式开源项目，提炼出它们在分层设计、驱动抽象、板级支持和构建系统方面的核心模式。这些模式经过大规模社区验证，具有极高的工程实践价值。

2.1 PX4 Autopilot

PX4是无人机领域最成熟的开源飞控软件之一，其架构设计堪称嵌入式系统的教科书。PX4的核心架构思想是"模块化+消息总线"：每个功能模块（driver、module、commander、navigator等）都是独立的编译单元，模块之间通过uORB（Micro Object Request Broker）消息总线进行通信，实现了真正的松耦合。

在目录结构上，PX4采用了清晰的功能分区：boards/存放所有板级支持包，每个板子一个子目录，包含初始化代码和引脚配置；src/drivers/存放所有设备驱动，按设备类型（imu、baro、mag、gps等）组织；src/modules/存放功能模块（姿态估计、位置控制等）；platforms/存放平台相关代码（nuttx、qurt、posix）。这种组织方式使得添加新板子或新驱动变得非常直观。

PX4的驱动模型特别值得借鉴：每个驱动都实现cdev（字符设备）接口，通过文件描述符进行访问。驱动在初始化时向uORB发布（advertise）主题（topic），应用模块通过订阅（subscribe）主题来获取数据。这种发布-订阅模式彻底解耦了数据生产者和消费者，使得传感器替换对应用层完全透明。

2.2 Zephyr RTOS

Zephyr是Linux基金会主导的实时操作系统，其架构设计深受Linux内核影响。Zephyr最突出的设计是设备树（Device Tree）机制和驱动模型。设备树以声明式的方式描述硬件拓扑，将硬件配置从代码中完全分离出来。每个板子有一个.dts文件描述其上的总线、设备和引脚分配，驱动通过设备树宏来获取硬件信息，而不是硬编码。

Zephyr的驱动模型定义了统一的device结构体，所有驱动都必须实现init()、read()、write()、ioctl()等标准操作。驱动通过DEVICE_DEFINE()宏在编译时注册，系统在启动时自动遍历并初始化所有已注册的设备。这种设计使得驱动的添加和移除完全由构建配置控制，无需修改应用代码。

在构建系统方面，Zephyr使用Kconfig进行细粒度的功能裁剪，配合CMake实现多目标构建。开发者只需指定-DBOARD=stm32g431kb即可自动选择正确的工具链、链接脚本和驱动配置。这种"配置即代码"的理念极大地简化了多平台支持的工作量。

2.3 RT-Thread

RT-Thread是国内最活跃的开源RTOS，其架构设计在实用性和规范性之间取得了很好的平衡。RT-Thread的核心设计思想是"面向对象+C语言模拟"：通过结构体和函数指针模拟面向对象的继承和多态，在C语言的限制下实现了优雅的抽象。

RT-Thread定义了丰富的设备类型：rt_device是所有设备的基类，派生出rt_spi_device、rt_i2c_device、rt_serial_device等子类。每个设备类型都有标准化的操作接口（open/close/read/write/control）。BSP层按bsp/stm32/stm32g431-st-nucleo/这样的层级组织，芯片系列和具体板子分层清晰。

特别值得借鉴的是RT-Thread的"I/O设备模型"：所有设备都挂载在虚拟文件系统上，应用层通过rt_device_open("i2c1")这样的统一接口访问设备，无需关心底层实现。这种设计在保持C语言效率的同时，提供了接近高级语言的开发体验。

2.4 ESP-IDF

ESP-IDF是乐鑫官方的物联网开发框架，其最突出的架构特点是组件化（Component-based）。ESP-IDF将所有功能模块都抽象为组件，每个组件拥有独立的CMakeLists.txt、Kconfig和头文件，可以独立编译、测试和版本管理。组件之间通过显式依赖声明（REQUIRES和PRIV_REQUIRES）建立关系，构建系统自动解析依赖拓扑并确定编译顺序。

ESP-IDF的组件目录结构非常规范：每个组件包含include/（公共头文件）、src/（源文件）、test/（测试代码）和Kconfig（配置选项）。这种结构使得组件可以独立发布和复用，ESP-IDF的组件注册中心（Component Registry）已经积累了数千个第三方组件。对于我们的AHRS项目，这种组件化思想可以直接应用于传感器驱动和算法模块的管理。

2.5 ChibiOS

ChibiOS是一个设计精良的实时操作系统，其架构以严格的HAL/LL分层著称。ChibiOS将硬件相关代码分为三层：PAL（Port Abstraction Layer，端口抽象层）处理GPIO，LLD（Low Level Driver，底层驱动）处理具体外设寄存器，HAL（Hardware Abstraction Layer）提供统一的高级接口。这种三层抽象使得同一套驱动代码可以支持同一芯片系列的不同型号，只需替换LLD层即可。

ChibiOS的板级配置采用board.h + board.mk的模式：board.h以宏定义的方式声明所有引脚分配和时钟配置，board.mk指定需要编译的源文件和包含路径。这种声明式的板级配置方式简洁高效，适合资源受限的嵌入式场景。

2.6 模式总结与提炼

综合以上五个项目的架构设计，我们可以提炼出以下核心模式，这些模式将作为我们新架构设计的理论基础：

模式	核心思想	代表项目
分层抽象	硬件/驱动/中间件/应用严格分层，层间通过接口交互	所有项目
设备接口统一	定义标准设备操作接口（init/read/write/ioctl），驱动实现接口	PX4/Zephyr/RT-Thread
总线抽象	I2C/SPI/UART等总线抽象为统一接口，传感器驱动依赖总线接口而非具体MCU	Zephyr/RT-Thread/ChibiOS
板级数据驱动	板级配置以声明式数据描述，而非硬编码在驱动中	Zephyr/ChibiOS/PX4
组件化/模块化	每个功能模块独立编译，显式声明依赖关系	ESP-IDF/PX4
构建配置化	通过Kconfig等机制实现功能裁剪和平台选择	Zephyr/ESP-IDF/RT-Thread
发布-订阅解耦	数据生产者与消费者通过消息总线解耦	PX4/uORB

3 分层架构设计

基于对五个顶级开源项目的深入研究，结合AHRs项目的实际需求（多MCU平台支持、多传感器兼容、算法模块化），我们设计了一个六层架构体系。这个架构的核心原则是：每一层只依赖其下层的抽象接口，绝不跨层依赖；同一层内的模块通过标准接口交互，实现可插拔替换。

3.1 六层架构总览

整个系统从下到上分为六层，每层有明确的职责边界和接口契约。下层为上层提供服务，上层通过标准接口调用下层功能，绝不直接访问下层的内部实现。这种严格的分层确保了修改某一层的实现不会波及其他层，是系统可维护性和可移植性的根本保障。

层级	名称	职责	典型内容	典型文件
L0	BSP层	板级支持包	引脚分配、时钟配置、板载设备声明	board.h / board_config.h

L1	芯片支持层	MCU芯片支持	芯片HAL封装、启动代码、链接脚本	chip_xxx_hal.c / startup.s
L2	驱动抽象层	硬件抽象接口	总线抽象(I2C/SPI/UART)、GPIO/Timer抽象	bus.h / drv_xxx.h
L3	传感器驱动层	设备驱动实现	IMU/MAG/BARO等传感器驱动	icm42688.c / mmc5603.c
L4	中间件层	通用功能模块	AHRS算法、滤波器、Shell、文件系统	mahony.c / shell.c
L5	应用层	业务逻辑	任务调度、Shell命令、主控逻辑	main.c / shell_cmds.c

依赖关系遵循严格的单向原则：L5依赖L4，L4依赖L3，以此类推。L3的传感器驱动只依赖L2的总线抽象接口，不直接调用L1的芯片HAL函数。L0的板级配置通过数据结构传递给L2和L3，而不是通过函数调用。这种依赖结构确保了：当你更换MCU平台时，只需替换L0和L1，L2-L5的代码完全不需要修改；当你更换传感器型号时，只需替换L3的对应驱动，L4-L5的代码完全不需要修改。

3.2 BSP层 (Board Support Package)

BSP层是整个架构的根基，它定义了一块具体电路板的所有硬件信息。BSP层的核心设计原则是"数据驱动"：板级信息以数据结构（而非函数）的形式描述，上层代码通过读取这些数据来配置硬件。这种设计使得添加新板子只需要新增一个数据描述文件，无需修改任何驱动代码。

每个BSP目录包含以下核心文件：board.h定义板级标识、板载设备列表和引脚分配表；board_config.h定义时钟配置和电源管理参数；board.c实现板级初始化函数，负责根据board.h中的数据配置所有GPIO和外设时钟。CubeMX生成的代码被隔离在cube/子目录中，作为参考但不直接参与编译，避免生成代码覆盖问题。

BSP层的关键数据结构是board_dev_desc_t（板载设备描述符），它声明了这块板子上有哪些设备、每个设备连接在哪条总线上、使用哪些引脚。传感器驱动在初始化时，通过设备描述符获取总线实例和引脚信息，而不是硬编码。这种设计使得同一块传感器芯片在不同板子上的引脚分配可以完全不同，驱动代码却完全相同。

3.3 芯片支持层 (Chip Support)

芯片支持层封装了特定MCU芯片的硬件细节，为上层提供统一的芯片级API。这一层的设计参考了ChibiOS的PAL/LLD分层和RT-Thread的BSP分层思想。芯片支持层按芯片系列组织，同一系列的不同型号共享大部分代码，只在LLD层有差异。

芯片支持层的目录按chips/stm32/g4xx/这样的层级组织。最内层包含：CMSIS核心文件和启动代码、芯片厂商HAL库的封装（不是直接使用厂商HAL，而是封装为统一接口）、链接脚本和Flash分区定义。对于ESP32这类自带SDK的平台，芯片支持层主要封装ESP-IDF的API，使其符合我们的统一接口规范。

特别需要注意的是，芯片支持层对外暴露的接口必须是MCU无关的。例如，我们定义`chip_gpio_write(pin, val)`这样的统一接口，在STM32上它内部调用`HAL_GPIO_WritePin()`，在ESP32上它内部调用`gpio_set_level()`。上层代码只调用`chip_gpio_write()`，完全不感知底层MCU的差异。

3.4 驱动抽象层 (Driver HAL)

驱动抽象层是整个架构中最关键的一层，它定义了所有硬件外设的抽象接口。这一层只有头文件和接口定义，没有具体实现（具体实现由芯片支持层提供）。驱动抽象层的设计直接参考了Zephyr的驱动模型和RT-Thread的设备驱动框架。

驱动抽象层的核心是三大类接口：总线接口（`bus_i2c_t`、`bus_spi_t`、`bus_uart_t`）定义了I2C/SPI/UART总线的标准操作，包括传输、读写寄存器等；GPIO接口（`drv_gpio_t`）定义了GPIO的标准操作，包括读写、中断注册等；定时器接口（`drv_timer_t`）定义了定时器的标准操作，包括启停、频率设置、回调注册等。

每个接口都采用函数指针结构体的方式定义，这是C语言实现多态的经典模式。以总线接口为例，`bus_i2c_t`结构体包含`init()`、`transfer()`、`read_reg()`、`write_reg()`等函数指针。STM32的实现填充这些指针为STM32 HAL函数的封装，ESP32的实现填充为ESP-IDF函数的封装。传感器驱动只持有`bus_i2c_t*`指针，调用其方法时完全不感知底层MCU。

3.5 传感器驱动层 (Sensor Drivers)

传感器驱动层实现了具体传感器芯片的驱动代码。这一层的设计核心是"驱动只依赖接口，不依赖平台"：每个传感器驱动只引用驱动抽象层定义的总线接口和GPIO接口，绝不直接调用任何MCU特定的API。这种设计使得传感器驱动具有完全的平台无关性，同一份ICM-42688驱动代码可以在STM32、AT32和ESP32上运行，无需任何修改。

传感器驱动按设备类型组织：`sensors/imu/`存放所有IMU驱动，`sensors/mag/`存放所有磁力计驱动，`sensors/baro/`存放所有气压计驱动。每种设备类型都有一个统一的接口头文件（如`imu.h`），定义了该类设备的标准操作和数据结构。具体驱动（如`icm42688.c`）实现这个接口，并通过注册机制将自己注册到系统中。

每个传感器驱动目录包含四个文件：驱动实现（`icm42688.c`）、驱动头文件（`icm42688.h`）、寄存器定义（`icm42688_reg.h`）和芯片数据手册（`datasheet.pdf`）。寄存器定义文件只包含寄存器地址和位域的宏定义，不包含任何逻辑代码，这种分离使得寄存器定义可以被其他工具（如Python校准脚本）复用。

3.6 中间件层 (Middleware)

中间件层包含与硬件无关的通用功能模块，这些模块只依赖传感器驱动层提供的标准接口，不依赖任何具体的传感器型号或MCU平台。中间件层是整个系统中复用价值最高的部分，因为它完全与硬件解耦。

对于AHRS项目，中间件层包含以下核心模块：AHRS算法模块（Mahony滤波、Madgwick滤波、EKF等）实现姿态解算，输入为标准化的IMU数据结构，输出为四元数/欧拉角；数字滤波器模块（低通滤波、卡尔曼滤波、滑动平均等）提供信号处理能力；Shell框架模块提供命令行交互能力；文件系统模块提供文件读写抽象；通信协议模块（MAVLink、自定义协议等）提供数据传输能力。

中间件层的设计原则是"零硬件依赖"：所有中间件模块都可以在PC上编译和单元测试。例如，AHRS算法模块可以在PC上用预先录制的IMU数据进行验证，无需实际硬件。这种可测试性极大地提高了开发效率和代码质量。PX4的SITL（Software In The Loop）仿真正是基于这种设计理念实现的。

3.7 应用层 (Application)

应用层是系统的最顶层，负责业务逻辑的编排和任务调度。应用层通过中间件层和传感器驱动层的标准接口获取数据和调用功能，不直接访问任何底层API。应用层的代码应该是项目特定的，不具备跨项目复用性，但它应该足够薄，只包含业务编排逻辑，所有通用功能都下沉到中间件层。

应用层包含：主入口（main.c）负责系统初始化和任务创建；任务模块（tasks/）实现各个RTS任务的主体逻辑；Shell命令模块（shell_cmds/）实现调试和配置命令。应用层的初始化流程遵循固定顺序：BSP初始化 -> 芯片支持层初始化 -> 驱动抽象层初始化 -> 传感器驱动注册与初始化 -> 中间件初始化 -> 创建应用任务。

4 C语言接口抽象设计

C语言没有原生的面向对象支持，但通过函数指针结构体、依赖注入和注册-发现机制，我们可以在C语言中实现优雅接口抽象。本章详细阐述这些核心设计模式的实施方案。

4.1 总线抽象接口

总线抽象是整个驱动框架的基石。传感器驱动通过总线抽象接口与MCU外设交互，完全不感知底层MCU的差异。我们为I2C、SPI和UART三种总线分别定义抽象接口，每种总线接口都包含初始化、传输和寄存器操作等标准方法。

I2C总线接口的定义如下，它包含了传感器驱动所需的所有I2C操作。transfer()是最基础的原子操作，其他方法（如read_reg()和write_reg()）可以基于transfer()实现，也可以由具体平台用更高效的方式覆盖：

```
typedef struct bus_i2c {
    int (*init)(struct bus_i2c *bus);
    int (*deinit)(struct bus_i2c *bus);
    int (*transfer)(struct bus_i2c *bus, uint16_t dev_addr,
        const uint8_t *wbuf, uint16_t wlen,
        uint8_t *rbuf, uint16_t rlen);
    int (*read_reg)(struct bus_i2c *bus, uint16_t dev_addr,
        uint8_t reg, uint8_t *buf, uint16_t len);
}
```

```
int (*write_reg)(struct bus_i2c *bus, uint16_t dev_addr,
uint8_t reg, const uint8_t *buf, uint16_t len);
void *priv; /* 平台私有数据 */
} bus_i2c_t;
```

SPI总线接口与I2C类似，但增加了片选控制和时钟极性/相位配置。SPI接口支持两种传输模式：单次传输（transfer()）和分段传输（transfer_seg()），后者允许在一次片选有效期间发送多个数据段，适用于需要连续读取大量寄存器的场景：

```
typedef struct bus_spi {
int (*init)(struct bus_spi *bus);
int (*deinit)(struct bus_spi *bus);
int (*transfer)(struct bus_spi *bus, uint8_t cs_pin,
const uint8_t *wbuf, uint16_t wlen,
uint8_t *rbuf, uint16_t rlen);
int (*transfer_seg)(struct bus_spi *bus, uint8_t cs_pin,
const uint8_t *wbuf1, uint16_t len1,
const uint8_t *wbuf2, uint16_t len2,
uint8_t *rbuf, uint16_t rlen);
int (*set_speed)(struct bus_spi *bus, uint32_t speed_hz);
void *priv;
} bus_spi_t;
```

4.2 设备接口模式

每种传感器类型都定义了统一的设备接口，所有该类型的驱动都必须实现这个接口。设备接口采用"结构体+函数指针"的模式，配合priv指针实现实例化。这种设计允许同一系统中同时存在多个同类型传感器的实例（例如双IMU冗余设计），每个实例拥有独立的私有数据。

以IMU设备接口为例，它定义了IMU传感器必须提供的所有操作：init()初始化传感器并返回操作结果；read_accel()读取三轴加速度数据；read_gyro()读取三轴角速度数据；read_temp()读取芯片温度；self_test()执行传感器自检；set_range()动态调整量程。所有数据都以SI单位（m/s²、rad/s、度）返回，驱动内部负责从原始值到SI单位的转换：

```
typedef struct imu_dev {
const char *name;
int (*init)(struct imu_dev *dev);
int (*deinit)(struct imu_dev *dev);
int (*read_accel)(struct imu_dev *dev, float accel[3]);
int (*read_gyro)(struct imu_dev *dev, float gyro[3]);
int (*read_temp)(struct imu_dev *dev, float *temp);
int (*self_test)(struct imu_dev *dev);
int (*set_range)(struct imu_dev *dev,
imu_accel_range_t ar, imu_gyro_range_t gr);
void *priv; /* 驱动私有数据 */
} imu_dev_t;
```

磁力计和气压计接口遵循相同的设计模式。磁力计接口（mag_dev_t）提供read_mag()读取三轴磁场强度，单位为高斯（Gauss）；气压计接口（baro_dev_t）提供read_pressure()和read_temp()，单位分别为帕斯卡和摄氏度。所有接口都遵循"统一单位、统一返回值、统一错误码"的三统一原则。

4.3 设备注册与发现机制

设备注册与发现机制使得应用层代码可以在运行时找到所需的设备实例，而不需要在编译时硬编码设备名称。这种机制借鉴了Linux内核的设备驱动模型和RT-Thread的设备注册框架。

设备注册采用链表+名称查找的方式。每个设备在初始化时调用device_register()将自己注册到全局设备表中，应用层通过device_find()按名称或类型查找设备实例。这种设计支持多个同类型设备共存，例如系统中有两个IMU时，可以通过名称"imu0"和"imu1"分别访问：

```
/* 设备类型枚举 */
typedef enum {
    DEV_TYPE_IMU = 0,
    DEV_TYPE_MAG,
    DEV_TYPE_BARO,
    DEV_TYPE_GPS,
    DEV_TYPE_UNKNOWN,
} device_type_t;

/* 设备注册表项 */
typedef struct device_node {
    const char *name;
    device_type_t type;
    void *dev; /* 指向具体设备结构体 */
    struct device_node *next;
} device_node_t;

/* 注册与发现API */
int device_register(const char *name, device_type_t type, void *dev);
void *device_find(const char *name);
void *device_find_by_type(device_type_t type, int index);
```

4.4 回调与事件机制

在实时系统中，数据获取通常有两种模式：轮询模式和中断/回调模式。对于AHRS系统，我们推荐使用定时回调模式：应用层注册一个定时回调函数，传感器驱动在数据就绪时（由定时器或DRDY中断触发）调用该回调，将数据推送给应用层。这种模式既避免了轮询的CPU浪费，又比纯中断模式更容易控制时序。

回调机制通过imu_data_cb_t等函数指针类型定义。应用层在初始化时通过imu_dev.register_callback()注册回调，驱动在每次数据更新时调用该回调。回调函数的参数包含时间戳、传感器数据和数据质量标志，应用层据此进行后续处理。这种设计参考了PX4的uORB订阅机制，但更加轻量，适合裸机或轻量级RTOS场景。

5 完整目录结构

综合以上架构设计，我们给出完整的目录结构方案。这个结构遵循"按功能分层、按类型分目录"的原则，每一层都有清晰的职责边界，每个目录都有明确的归属规则。

```
ahrs/
├── boards/ # L0: BSP层 - 板级支持包
│   ├── stm32g431kbu6/ # STM32G431开发板
│   │   ├── board.h # 板级定义(引脚/设备表)
│   │   ├── board.c # 板级初始化实现
│   │   ├── board_config.h # 时钟/电源配置
│   │   └── cube/ # CubeMX生成代码(仅供参考)
│   │       ├── Core/
│   │       ├── Drivers/
│   │       └── IMU.ioc
│   ├── linker/ # 链接脚本
│   │   └── STM32G431XX_FLASH.ld
│   ├── startup/ # 启动代码
│   │   └── startup_stm32g431xx.s
│   ├── at32f435/ # AT32F435开发板
│   │   ├── board.h
│   │   ├── board.c
│   │   └── ...
│   └── esp32s3/ # ESP32-S3开发板
│       ├── board.h
│       ├── board.c
│       └── ...
├── chips/ # L1: 芯片支持层
│   ├── stm32/ # STM32芯片系列
│   │   ├── g4xx/ # G4系列
│   │   │   ├── hal/ # HAL封装
│   │   │   │   ├── chip_gpio.c
│   │   │   │   ├── chip_i2c.c
│   │   │   │   ├── chip_spi.c
│   │   │   │   ├── chip_uart.c
│   │   │   │   ├── chip_timer.c
│   │   │   └── chip_flash.c
│   │   ├── cmsis/ # CMSIS核心
│   │   └── STM32G4xx_HAL_Driver/# 原厂HAL库
│   ├── f4xx/ # F4系列
│   │   └── ...
│   ├── at32/ # AT32芯片系列
│   │   ├── f435/
│   │   └── ...
│   ├── esp32/ # ESP32芯片系列
│   ├── esp32s3/
│   └── ...
├── drivers/ # L2+L3: 驱动层
│   ├── hal/ # L2: 驱动抽象层(接口定义)
│   │   ├── bus.h # 总线抽象接口
│   │   ├── bus_i2c.h # I2C总线接口
│   │   ├── bus_spi.h # SPI总线接口
│   │   ├── bus_uart.h # UART总线接口
│   │   └── drv_gpio.h # GPIO抽象接口
```

```

| | └─ drv_timer.h # Timer抽象接口
| | └─ drv_flash.h # Flash抽象接口
| | └─ drv_crc.h # CRC抽象接口
| | └─ drv_usb.h # USB抽象接口
|
| └─ sensors/ # L3: 传感器驱动层
|   └─ imu/ # IMU传感器
|     └─ imu.h # IMU统一接口
|     └─ icm42688/ # ICM-42688驱动
|       └─ icm42688.c
|       └─ icm42688.h
|       └─ icm42688_reg.h
|       └─ datasheet.pdf
|     └─ mpu6050/ # MPU6050驱动
|     └─ bmi088/ # BMI088驱动
|   └─ mag/ # 磁力计
|     └─ mag.h # 磁力计统一接口
|     └─ mmc5603/
|     └─ mmc5983/
|   └─ baro/ # 气压计
|     └─ baro.h # 气压计统一接口
|     └─ spl06001/
|
| └─ middleware/ # L4: 中间件层
|   └─ ahrs/ # AHRs算法
|     └─ ahrs.h # AHRs统一接口
|     └─ mahony.c # Mahony滤波
|     └─ madgwick.c # Madgwick滤波
|     └─ ekf/ # 扩展卡尔曼滤波
|       └─ ekf.c
|       └─ ekf.h
|     └─ filter/ # 数字滤波器
|       └─ lowpass.c
|       └─ kalman.c
|       └─ moving_avg.c
|     └─ shell/ # Shell框架
|       └─ shell_port.c # Shell移植接口
|       └─ shell_port.h
|     └─ fs/ # 文件系统抽象
|     └─ protocol/ # 通信协议
|     └─ mavlink/
|     └─ custom/
|
| └─ app/ # L5: 应用层
|   └─ main.c # 主入口
|   └─ tasks/ # 应用任务
|     └─ task_ahrs.c # AHRs计算任务
|     └─ task_sensor.c # 传感器采集任务
|     └─ task_telemetry.c # 遥测任务
|   └─ shell_cmds/ # Shell命令
|   └─ cmd_sensor.c
|   └─ cmd_flash.c
|   └─ cmd_ahrs.c

```

```
├── libs/ # 第三方库
│   ├── letter-shell/
│   │   ├── src/
│   │   ├── LICENSE
│   │   └── CMakeLists.txt
│   ├── lwrb/
│   │   ├── lwrb.c
│   │   ├── lwrb.h
│   │   └── CMakeLists.txt
│   ├── printf/
│   │   ├── printf.c
│   │   ├── printf.h
│   │   └── CMakeLists.txt
│   └──
├── tools/ # 工具与脚本
│   ├── python/ # Python工具
│   │   ├── data_logger.py # 数据记录
│   │   ├── calibration.py # 传感器校准
│   │   └── plot_imu.py # IMU数据可视化
│   ├── scripts/ # 构建脚本
│   │   ├── fix_cubemx.py
│   │   └── flash.sh
│   ├── configs/ # IDE/调试配置
│   ├── openocd/
│   └── jlink/
├── docs/ # 文档
│   └── architecture.md
├── CMakeLists.txt # 根CMake
├── Kconfig # 根配置
├── Makefile # 顶层Makefile(委托CMake)
└── README.md
```

6 构建系统设计

构建系统是多平台支持的关键基础设施。我们设计了一套基于CMake的多目标构建系统，配合Kconfig风格的配置机制，实现"一次配置、多平台编译"的能力。

6.1 CMake多目标构建

构建系统的核心思想是"目标驱动"：每个板子定义一个CMake预设（Preset），指定工具链、芯片系列、板级目录和需要编译的驱动/中间件模块。开发者只需执行`cmake --preset=stm32g431`即可自动配置整个构建环境。

根`CMakeLists.txt`负责解析目标预设，加载对应的工具链文件、芯片支持代码、BSP代码和驱动/中间件模块。每个模块（传感器驱动、中间件模块、第三方库）都有自己的`CMakeLists.txt`，通过条件编

译（基于Kconfig配置）决定是否参与构建。这种设计参考了ESP-IDF的组件化构建和Zephyr的模块化CMake。

典型的构建命令如下：`cmake --preset=stm32g431`配置STM32G431目标；`cmake --preset=esp32s3`配置ESP32-S3目标；`cmake --preset=at32f435`配置AT32F435目标。每个预设自动选择正确的交叉编译工具链、链接脚本和启动代码，开发者无需手动配置任何路径。

6.2 Kconfig配置系统

Kconfig是Linux内核社区验证过的大规模配置管理方案，Zephyr和ESP-IDF都采用了它。Kconfig的核心优势是：配置项之间可以声明依赖关系（例如启用ICM-42688驱动自动启用I2C总线支持），配置项可以有类型检查和取值范围约束，配置结果以头文件的形式输出，供C代码条件编译使用。

对于AHRs项目，我们定义以下主要配置项：芯片系列选择（`CONFIG_CHIP_STM32G4XX`等）、板级选择（`CONFIG_BOARD_STM32G431KBU6`等）、传感器驱动选择（`CONFIG_SENSOR_ICM42688`等）、中间件模块选择（`CONFIG_AHRs_MAHONY`等）和功能选项（`CONFIG_DEBUG_TRACE`等）。每个板子的预设文件中包含一组默认配置值，开发者可以通过menuconfig界面进行自定义调整。

```
# Kconfig - 传感器驱动配置

menu "Sensor Drivers"

config SENSOR_ICM42688
bool "ICM-42688 IMU sensor"
select BUS_I2C
default n
help
Enable support for TDK ICM-42688 6-axis IMU.
This sensor provides accelerometer and gyroscope data.
Requires I2C bus support.

config SENSOR_MMC5603
bool "MMC5603 magnetometer"
select BUS_I2C
default n
help
Enable support for Memsic MMC5603 magnetometer.

config SENSOR_SPL06001
bool "SPL06-001 barometer"
select BUS_I2C
default n
help
Enable support for Goertek SPL06-001 barometer.

endmenu
```

7 关键代码模板

本章给出三个关键代码模板，展示架构设计的具体实现方式。这些模板可以直接作为项目开发的起点。

7.1 总线抽象实现

以下代码展示了STM32平台上I2C总线抽象的实现。这个实现将STM32 HAL的I2C API封装为我们定义的bus_i2c_t接口，使得上层传感器驱动可以通过统一接口访问I2C总线：

```
/* chips/stm32/g4xx/hal/chip_i2c.c */

#include "bus_i2c.h"
#include "stm32g4xx_hal.h"

typedef struct {
    I2C_HandleTypeDef *hi2c;
} stm32_i2c_priv_t;

static int stm32_i2c_init(bus_i2c_t *bus)
{
    stm32_i2c_priv_t *priv = bus->priv;
    /* HAL_I2C_Init已在MX_I2C_Init中完成 */
    return (HAL_I2C_IsDeviceReady(priv->hi2c,
    0x00, 3, 100) == HAL_OK) ? 0 : -1;
}

static int stm32_i2c_transfer(bus_i2c_t *bus,
uint16_t dev_addr, const uint8_t *wbuf, uint16_t wlen,
uint8_t *rbuf, uint16_t rlen)
{
    stm32_i2c_priv_t *priv = bus->priv;
    HAL_StatusTypeDef ret;
    if (wlen > 0 && rlen > 0) {
        ret = HAL_I2C_Mem_Read(priv->hi2c,
        dev_addr << 1, wbuf[0],
        I2C_MEMADD_SIZE_8BIT, rbuf, rlen, 1000);
    } else if (wlen > 0) {
        ret = HAL_I2C_Master_Transmit(priv->hi2c,
        dev_addr << 1, wbuf, wlen, 1000);
    } else {
        ret = HAL_I2C_Master_Receive(priv->hi2c,
        dev_addr << 1, rbuf, rlen, 1000);
    }
    return (ret == HAL_OK) ? 0 : -1;
}

static int stm32_i2c_read_reg(bus_i2c_t *bus,
uint16_t dev_addr, uint8_t reg,
uint8_t *buf, uint16_t len)
{
    stm32_i2c_priv_t *priv = bus->priv;
    return (HAL_I2C_Mem_Read(priv->hi2c,
    dev_addr << 1, reg,
```

```

I2C_MEMADD_SIZE_8BIT,
buf, len, 1000) == HAL_OK) ? 0 : -1;
}

static int stm32_i2c_write_reg(bus_i2c_t *bus,
uint16_t dev_addr, uint8_t reg,
const uint8_t *buf, uint16_t len)
{
    stm32_i2c_priv_t *priv = bus->priv;
    return (HAL_I2C_Mem_Write(priv->hi2c,
dev_addr << 1, reg,
I2C_MEMADD_SIZE_8BIT,
(uint8_t*)buf, len, 1000) == HAL_OK) ? 0 : -1;
}

/* 工厂函数: 创建STM32 I2C总线实例 */
bus_i2c_t *stm32_i2c_create(I2C_HandleTypeDef *hi2c)
{
    static stm32_i2c_priv_t priv;
    static bus_i2c_t bus = {
        .init = stm32_i2c_init,
        .transfer = stm32_i2c_transfer,
        .read_reg = stm32_i2c_read_reg,
        .write_reg = stm32_i2c_write_reg,
        .priv = &priv,
    };
    priv.hi2c = hi2c;
    return &bus;
}

```

7.2 IMU驱动模板

以下代码展示了ICM-42688驱动如何基于总线抽象接口实现。注意驱动代码中没有任何STM32或ESP32的API调用，只通过bus_i2c_t接口访问硬件，因此具有完全的平台无关性：

```

/* drivers/sensors/imu/icm42688/icm42688.c */

#include "imu.h"
#include "icm42688.h"
#include "icm42688_reg.h"
#include "bus_i2c.h"

typedef struct {
    bus_i2c_t *bus; /* 总线实例(由BSP注入) */
    uint16_t addr; /* I2C设备地址 */
    float accel_scale;
    float gyro_scale;
} icm42688_priv_t;

static int icm42688_init(imu_dev_t *dev)
{
    icm42688_priv_t *priv = dev->priv;

```

```

uint8_t whoami;
/* 读取芯片ID验证通信 */
if (priv->bus->read_reg(priv->bus, priv->addr,
ICM42688_WHO_AM_I, &whoami, 1) != 0)
return -1;
if (whoami != ICM42688_WHO_AM_I_VAL)
return -2;
/* 软复位 */
uint8_t rst = 0x01;
priv->bus->write_reg(priv->bus, priv->addr,
ICM42688_REG_BANK_SEL, &rst, 1);
/* 配置加速度计和陀螺仪 ... */
return 0;
}

static int icm42688_read_accel(imu_dev_t *dev, float accel[3])
{
icm42688_priv_t *priv = dev->priv;
uint8_t raw[6];
if (priv->bus->read_reg(priv->bus, priv->addr,
ICM42688_ACCEL_XOUT_H, raw, 6) != 0)
return -1;
int16_t x = (raw[0] << 8) | raw[1];
int16_t y = (raw[2] << 8) | raw[3];
int16_t z = (raw[4] << 8) | raw[5];
accel[0] = x * priv->accel_scale;
accel[1] = y * priv->accel_scale;
accel[2] = z * priv->accel_scale;
return 0;
}

/* ... read_gyro, read_temp, self_test ... */

/* 创建ICM-42688设备实例 */
imu_dev_t *icm42688_create(bus_i2c_t *bus, uint16_t addr)
{
static icm42688_priv_t priv = {0};
static imu_dev_t dev = {
.name = "icm42688",
.init = icm42688_init,
.read_accel = icm42688_read_accel,
.read_gyro = icm42688_read_gyro,
.read_temp = icm42688_read_temp,
.self_test = icm42688_self_test,
.set_range = icm42688_set_range,
.priv = &priv,
};
priv.bus = bus;
priv.addr = addr;
return &dev;
}

```

7.3 BSP板级配置

以下代码展示了BSP层的板级配置，它以数据驱动的方式声明板上的设备和引脚分配。应用层和驱动层通过这些数据来配置硬件，而不是硬编码：

```
/* boards/stm32g431kbu6/board.h */

#ifndef BOARD_H
#define BOARD_H

#include "bus_i2c.h"
#include "bus_spi.h"
#include "drv_gpio.h"

/* — 板级标识 — */
#define BOARD_NAME "stm32g431kbu6"
#define BOARD_MCU "stm32g431kbu6"

/* — LED引脚定义 — */
#define LED0_PIN PIN('B', 3)
#define LED1_PIN PIN('B', 4)

/* — 按键引脚定义 — */
#define BTN0_PIN PIN('C', 13)

/* — 传感器引脚定义 — */
#define ICM42688_INT_PIN PIN('A', 0)
#define MMC5603_INT_PIN PIN('A', 1)

/* — 板载设备描述符 — */
typedef struct {
    const char *name;
    device_type_t type;
    bus_i2c_t *bus; /* 所属总线实例 */
    uint16_t i2c_addr; /* I2C设备地址 */
    uint8_t int_pin; /* 中断引脚 */
} board_dev_desc_t;

/* 板载设备表 */
static const board_dev_desc_t board_devices[] = {
{
    .name = "icm42688",
    .type = DEV_TYPE_IMU,
    .bus = &bus_i2c1, /* 由board.c提供 */
    .i2c_addr = 0x68,
    .int_pin = ICM42688_INT_PIN,
},
{
    .name = "mmc5603",
    .type = DEV_TYPE_MAG,
    .bus = &bus_i2c1,
    .i2c_addr = 0x30,
    .int_pin = MMC5603_INT_PIN,
},
}
```

```
{ NULL, DEV_TYPE_UNKNOWN, NULL, 0, 0 } /* 终止符 */  
};  
  
/* — 板级初始化API — */  
int board_init(void);  
const board_dev_desc_t *board_get_devices(void);  
  
#endif /* BOARD_H */
```

8 迁移路径与实施建议

架构重构不是一蹴而就的，需要分阶段、有策略地推进。我们建议采用"渐进式迁移"策略，在保证项目可编译可运行的前提下，逐步将现有代码迁移到新架构中。

8.1 第一阶段：建立骨架（1-2周）

第一阶段的目标是建立新架构的目录骨架和核心接口定义，不涉及功能代码的迁移。具体工作包括：创建新的目录结构；编写驱动抽象层的头文件（bus.h、drv_gpio.h等）；编写设备接口头文件（imu.h、mag.h、baro.h）；编写设备注册机制的代码；搭建CMake多目标构建框架。这个阶段结束后，项目应该可以在新目录结构下编译通过（虽然功能还不完整）。

8.2 第二阶段：迁移驱动（2-3周）

第二阶段的目标是将现有的驱动代码迁移到新架构中。具体工作包括：将driver/hal/下的MCU特定代码迁移到chips/stm32/g4xx/hal/，并封装为统一接口；将driver/icm42688/等传感器驱动迁移到drivers/sensors/，改为基于总线抽象接口；将board/下的板级配置重构为数据驱动的board.h。这个阶段的关键是确保每个迁移后的驱动都能正常工作，建议每迁移一个驱动就进行功能验证。

8.3 第三阶段：迁移中间件与应用（1-2周）

第三阶段的目标是将中间件和应用代码迁移到新架构中。具体工作包括：将Shell框架迁移到middleware/shell/；将应用任务迁移到app/tasks/；将Shell命令迁移到app/shell_cmds/；将第三方库迁移到libs/并为每个库添加CMakeLists.txt。这个阶段结束后，项目应该在新架构下完全恢复功能。

8.4 第四阶段：扩展与优化（持续）

第四阶段是持续的扩展和优化工作，包括：添加新MCU平台支持（AT32、ESP32）；添加新传感器驱动；实现AHRS算法模块；添加Python工具链；完善Kconfig配置系统；编写单元测试。这个阶段没有固定的时间表，根据项目需求逐步推进。

在添加新MCU平台时，核心工作量在于chips/目录下的芯片支持层实现：需要为新MCU实现总线抽象接口（chip_i2c.c、chip_spi.c等）和GPIO/Timer等外设抽象。一旦芯片支持层完成，所有传感器驱动和中间件代码无需修改即可在新平台上运行。这就是良好架构设计的投资回报：前期多投入的架构设计工作，在后续每增加一个新平台时都会节省大量的重复劳动。

8.5 关键注意事项

在迁移过程中，有几个关键注意事项需要特别关注。首先，始终保持项目可编译可运行，不要一次性重写所有代码，而是采用"旧代码不动、新代码并行"的策略，逐步切换。其次，每完成一个阶段的迁移，都要进行完整的功能回归测试，确保没有引入回归缺陷。第三，充分利用Git的分支管理能力，每个阶段在独立分支上开发，完成后再合并到主分支。第四，文档同步更新，确保目录结构说明、接口文档和构建指南与代码保持一致。最后，不要追求完美，架构设计是一个持续演进的过程，先让系统跑起来，再逐步优化。